

# UAC Predictive Intelligence: A Machine Learning and Time-Series Framework for Care Load and Placement Demand Forecasting

Priyanshu Yadav

Department of Computer Applications, Galgotias University  
, India

Email: [py5303022@gmail.com](mailto:py5303022@gmail.com)

**Abstract**—Operational care-load planning requires timely estimates of future population levels, transfer pressure, and discharge demand. This paper presents the design and implementation of a predictive intelligence dashboard for forecasting daily HHS care load using a combination of supervised machine learning, seasonal time-series modeling, and scenario analysis. The implementation accepts comma-separated value data, standardizes heterogeneous column names, converts dates and numeric fields, aggregates duplicate dates, reindexes the observations to a daily calendar, and interpolates missing numeric values while preserving an explicit missing-date indicator. Temporal and operational predictors include lagged care-load values, rolling means and standard deviations, transfer and discharge lags, net pressure, and calendar variables. A Gradient Boosting Regressor is used as the primary feature-based model, while Seasonal Autoregressive Integrated Moving Average (SARIMA) and a persistence-based naive model provide statistical and baseline comparisons. Recursive multi-step forecasting, quantile-based prediction intervals, weekday-based discharge estimation, capacity thresholds, and intake stress scenarios are integrated into an interactive Streamlit dashboard. The supplied implementation does not include a dataset or stored experimental outputs; therefore, numerical accuracy claims are intentionally not fabricated. Instead, the paper documents the reproducible methodology, evaluation protocol, system architecture, and operational interpretation implemented by the code.

**Index Terms**—care-load forecasting, gradient boosting, machine learning, SARIMA, time-series forecasting, scenario analysis, Streamlit.

## I. INTRODUCTION

Forecasting operational care demand is a time-dependent prediction problem in which historical observations can contain persistence, short-term fluctuations, weekly patterns, and changes associated with transfers and discharges. A useful analytical system must therefore do more than fit a single predictive model: it should establish a reliable data pipeline, compare alternative forecasting strategies, quantify uncertainty, and expose results in a form that supports operational planning.

The presented system, named UAC Predictive Intelligence in the implementation, is an interactive forecasting dashboard that predicts future HHS care load and estimates short-term discharge demand. The source code implements

data validation, feature engineering, model evaluation, recursive forecasting, quantile intervals, scenario analysis, capacity monitoring, and visualization in a single Streamlit application. The application expects daily observations for apprehensions, CBP custody, transfers, HHS care, and discharges [1].

The main contributions of this work are:

- 1) a reproducible preprocessing pipeline for irregular daily operational data.
- 2) a feature-based Gradient Boosting forecasting approach using temporal and flow-related variables.
- 3) a SARIMA and naive baseline comparison under a chronological hold-out.
- 4) recursive scenario forecasting with transfer multipliers.
- 5) quantile-based uncertainty bands.
- 6) an interactive planning interface with capacity and discharge-demand indicators.

## II. RELATED WORK

Gradient boosting is a stage-wise additive modeling approach in which weak learners are fitted sequentially to improve a loss function. Friedman introduced the gradient boosting framework for regression and classification and demonstrated the effectiveness of tree-based boosting models [2]. The implementation uses the GradientBoostingRegressor available in scikit-learn, with 300 estimators, a learning rate of 0.05, tree depth 3, and a fixed random seed [1], [3].

Classical time-series methods remain important when temporal dependence and seasonality are strong. The implementation uses SARIMA with order (1,1,1) and seasonal order (1,1,1,7), representing a daily series with a weekly seasonal period. The SARIMAX interface in statsmodels supports autoregressive, differencing, moving-average, and seasonal components [4]. The system additionally includes a persistence baseline that repeats the latest observed value, providing a simple reference against which more complex models can be evaluated.

Interactive analytical applications can reduce the distance between predictive modeling and operational use. Streamlit is a Python framework for creating data applications and supports interactive widgets and data visualization [5]. In the present system, Streamlit is used to collect the CSV input, expose forecast controls, display metrics, and organize results into forecasting, discharge-demand, model-comparison, and data-quality tabs.

## III. SYSTEM DESIGN AND METHODOLOGY

### A. Input Data and Schema Standardization

The expected schema contains Date, Apprehended, CBP\_Custody, Transfers, HHS\_Care, and Discharged. Because source files may contain different column labels or footnote marks, the implementation normalizes headers by lowercasing, trimming whitespace, removing trailing symbols or digits, and matching aliases. If required fields remain absent, the application stops processing and reports the missing fields rather than silently proceeding [1].

### B. Data Cleaning and Temporal Regularization

Dates are parsed with error coercion and invalid dates are removed. Operational variables are converted to numeric values after removing comma separators. Duplicate dates are aggregated by the mean. The cleaned observations are then reindexed to a complete daily calendar. Importantly, missing calendar dates are not interpreted as zero observations. Instead, a `Was_Missing_Date` indicator is retained and numeric variables are interpolated in both directions. This design preserves the distinction between an absent record and an observed zero.

### C. Feature Engineering

The feature vector contains historical care-load values at one-, two-, seven-, and fourteen-day lags; seven- and fourteen-day rolling means and standard deviations; current transfer and discharge values; transfer and discharge lags; net pressure; and calendar variables. Net pressure is defined as:

$$\text{NetPressure}(t) = \text{Transfers}(t) - \text{Discharged}(t)$$

Calendar features are `DayOfWeek`, `Month`, `DayOfYear`, `WeekOfYear`, and `IsWeekend`. These variables allow the machine-learning model to represent both short-term temporal dependence and recurring calendar effects.

### D. Gradient Boosting Model

The primary machine-learning model is a `GradientBoostingRegressor` with `n_estimators = 300`, `learning_rate = 0.05`, `max_depth = 3`, and `random_state = 42`. The model is trained on complete feature rows after lag and rolling-window construction. During multi-step forecasting, predicted HHS care values are appended to the history so that later predictions can use earlier forecasts as lagged inputs.

### E. SARIMA and Naive Baselines

The statistical comparator is `SARIMA(1,1,1)×(1,1,1,7)`, fitted directly to the HHS care series. The weekly seasonal period of seven is consistent with daily observations and the weekly structure assumed by the application. If SARIMA fitting fails, the dashboard falls back to the naive forecast for display. The naive method repeats the last observed HHS care value for every future day and constructs a simple uncertainty band from the standard deviation of the latest fourteen observations.

### F. Model Evaluation

Evaluation is chronological rather than randomly shuffled. A configurable hold-out window between 7 and 30 days is taken from the end of the complete feature dataset, with the requested value capped according to the available observations. Three metrics are calculated:

Mean Absolute Error (MAE):  $\text{MAE} = (1/n) \sum |y_i - \hat{y}_i|$ .  
 Root Mean Squared Error (RMSE):  $\text{RMSE} = \sqrt{[(1/n) \sum (y_i - \hat{y}_i)^2]}$ .  
 Mean Absolute Percentage Error (MAPE):  $\text{MAPE} = (100/n) \sum |(y_i - \hat{y}_i)/y_i|$ , using only observations with  $|y_i| > 10^{-9}$ .

The dashboard sorts the resulting comparison table by MAE. This is an evaluation mechanism, not a claim that one model universally outperforms another. The actual ranking depends on the uploaded dataset and the selected hold-out window.

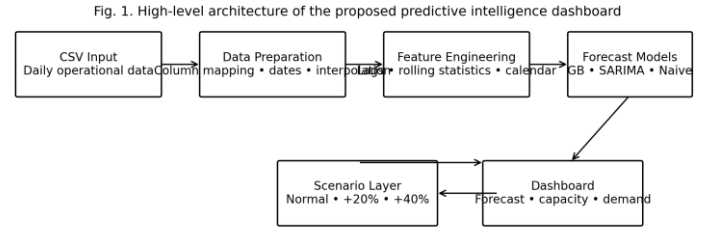


Fig. 1. High-level architecture of the implemented predictive intelligence system.

### G. Forecast Uncertainty and Scenario Analysis

For Gradient Boosting, the implementation trains three quantile regressors with alpha values 0.1, 0.5, and 0.9. Their predictions form lower, median, and upper forecast values. For SARIMA, an approximately 80% interval is obtained from the fitted model's confidence interval with alpha = 0.2. The dashboard presents these ranges as forecast uncertainty rather than guaranteed bounds.

The scenario layer changes the assumed transfer volume using multipliers of 1.0, 1.2, and 1.4, corresponding to Normal, High Intake (+20%), and Stress (+40%). Discharges are assumed from the recent seven-day average. The scenario calculations are applied to the Gradient Boosting model; the SARIMA and naive forecasts use historical series behavior without scenario transfer multipliers.

## IV. IMPLEMENTATION

The application is implemented in Python using pandas and NumPy for data processing, scikit-learn for Gradient Boosting and evaluation metrics, statsmodels for SARIMA, Plotly for interactive figures, and Streamlit for the web interface [1], [3]–[5]. The dashboard accepts CSV input and provides forecast horizons of 7, 14, or 30 days. Users can select the forecasting model, choose an intake scenario, and specify a planning capacity.

### A. Operational Indicators

The interface reports current HHS care, latest transfers, latest discharges, and net pressure. For the selected horizon it also reports the end-of-horizon forecast, peak forecast, and peak capacity utilization. A warning is raised when forecast care load exceeds the configured planning capacity; otherwise, a watch threshold of 90% is used.

### B. Placement and Discharge Demand

Discharge demand is estimated using a seasonal-naive method: recent discharges over a configurable 56-day lookahead are grouped by weekday, and the corresponding weekday mean is used for future dates. The dashboard sums

these daily values over the selected forecast horizon to provide an estimated total demand.

### C. Data Quality Monitoring

A dedicated tab exposes the number of records after daily reindexing, missing calendar dates, and missing numeric cells after interpolation. It also displays the most recent fourteen observations and their calculated net pressure. This makes data-quality issues visible alongside model outputs rather than treating preprocessing as an invisible step.

## V. EXPERIMENTAL EVALUATION AND RESULTS

The source code supplied for this study contains the complete evaluation procedure but does not contain the underlying CSV dataset or a saved run of the evaluation table. Consequently, this paper does not report numerical MAE, RMSE, or MAPE values that cannot be independently reproduced from the supplied materials. This is intentional: inserting numerical results without the corresponding observations would create unsupported experimental claims. When the dataset is supplied, the evaluation should be reported using the exact chronological hold-out implemented in the code

The code also supports direct visual comparison of actual and predicted values on the hold-out interval. A rigorous empirical study should report the dataset period, number of daily observations, number of missing dates, selected hold-out length, and the resulting metrics. If multiple datasets or rolling-origin evaluations are later used, the results should be reported separately rather than pooled without justification.

## VI. DISCUSSION

The implemented design has several methodological strengths. First, the chronological split avoids leakage that could occur if future observations were randomly mixed into the training set. Second, the feature-based model combines lagged care load with operational-flow variables, allowing the forecast to incorporate recent transfer and discharge conditions. Third, the inclusion of SARIMA and naive baselines prevents the evaluation from depending on a single model family. Fourth, scenario and capacity views translate forecasts into operational quantities that can be inspected interactively.

At the same time, several limitations should be considered. Interpolation can alter the temporal signal when observations are missing for long periods. The future transfer and discharge assumptions are deterministic in the current scenario engine, so scenario forecasts do not model uncertainty in those inputs. Recursive Gradient Boosting forecasts can accumulate error because predictions become inputs for later steps. The model also does not perform automated hyperparameter search, feature importance analysis, probabilistic calibration testing, or rolling-origin cross-validation.

The discharge-demand component is intentionally simpler than the HHS care-load model: it uses weekday averages over the recent history. This provides an interpretable planning estimate, but it does not capture changing covariate relationships or abrupt operational shifts. The application should therefore be treated as a decision-support system,

consistent with the explicit warning in the implementation that future transfers and discharges are uncertain [1].

## VII. CONCLUSION

This paper presented a reproducible predictive intelligence framework implemented as a Streamlit dashboard for daily HHS care-load and placement/discharge demand forecasting. The system combines structured data validation, temporal feature engineering, Gradient Boosting regression, SARIMA, a naive baseline, quantile uncertainty estimates, scenario analysis, capacity monitoring, and interactive visualization. Its evaluation design uses a chronological hold-out and MAE, RMSE, and MAPE to compare model behavior.

Because the supplied material contains code but no underlying dataset or recorded model outputs, quantitative performance claims are deliberately deferred until an actual experimental run is completed. Future work should include rolling-origin evaluation, hyperparameter optimization, feature-importance and explainability analysis, probabilistic calibration, uncertainty propagation for transfer and discharge assumptions, and evaluation across multiple time periods or datasets.

## ACKNOWLEDGMENT

The author acknowledges the open-source Python ecosystem, including scikit-learn, statsmodels, Streamlit, pandas, NumPy, and Plotly, which provides the computational and visualization components used by the implementation.

## REFERENCES

- [1] P. Yadav, "UAC Predictive Intelligence Dashboard," source-code implementation supplied for this study, 2026.
- [2] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001, doi: 10.1214/aos/1013203451.
- [3] Scikit-learn Developers, "GradientBoostingRegressor," scikit-learn documentation, 2026. [Online]. Available: <https://scikit-learn.org/>
- [4] Statsmodels Developers, "SARIMAX," statsmodels documentation, 2026. [Online]. Available: <https://www.statsmodels.org/>
- [5] Streamlit, "Streamlit Documentation," 2026. [Online]. Available: <https://docs.streamlit.io/>